

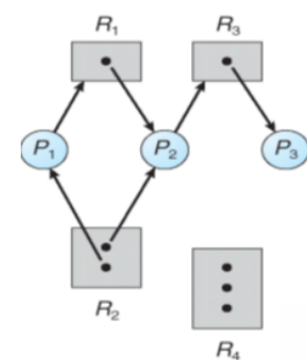
Deadlock – Condiciones y grafo de allocation

Condiciones para entrar en deadlock:

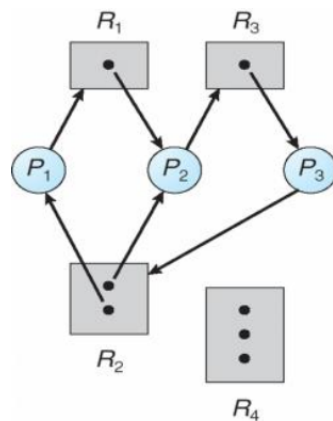
El deadlock puede cumplirse si se cumplen las cuatro condiciones simultaneamente

- ▶ **Exclusión Mutua** : solo un proceso a la vez puede usar un recurso.
- ▶ **Retener y Esperar**: un proceso mantiene al menos un recurso y está esperando adquirir recursos adicionales que tienen en tenencia otros procesos.
- ▶ **No Apropiación**: Un recurso puede ser liberado solo voluntariamente por el proceso que lo tiene, después que el proceso ha completado su tarea.
- ▶ **Espera Circular**: existe un conjunto $\{P_0, P_1, \dots, P_n\}$ de procesos esperando tal que P_0 está esperando por un recurso que es retenido por P_1 , P_1 está esperando por un recurso que es retenido por P_2 , ..., P_{n-1} está esperando por un recurso que es retenido por P_n , y P_n está esperando por un recurso que es retenido por P_0 .

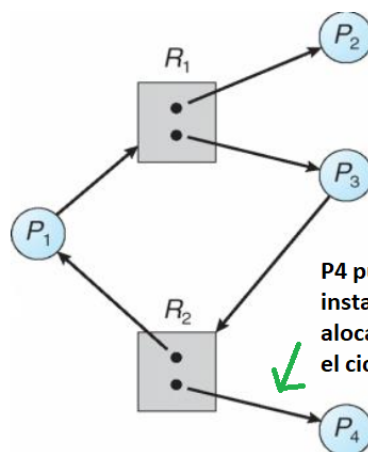
Grafo de allocation



a) Grafo sin ciclo



b) Grafo con interbloqueo



P4 puede liberar su instancia de R2, que seria asignada a P3, rompiendo el ciclo

c) Grafo con ciclo sin interbloqueo

- ▶ Si un grafo no contiene ciclos $\Rightarrow$ no hay interbloqueo.
- ▶ Si un grafo contiene un ciclo $\Rightarrow$
 - Si hay una sola instancia por tipo de recurso, entonces hay interbloqueo.
 - Si hay varias instancias por tipo de recurso, hay posibilidad de interbloqueo.

Ademas, si el ciclo involucra solo un conjunto de tipo de recursos, cada uno de los cuales tienen una sola instancia, entonces hay deadlock (cada proceso del ciclo esta en deadlock). En este caso, un ciclo en el grafo es tanto una condición necesaria como suficiente para que exista el deadlock.

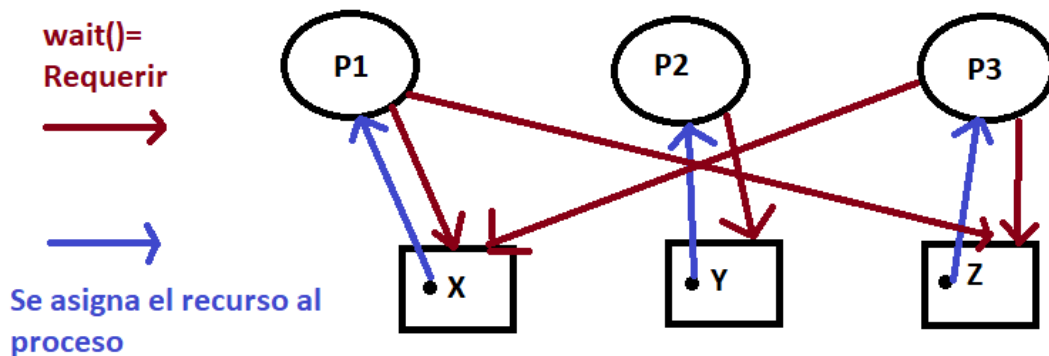
Si cada tipo de recurso tiene varias instancias, entonces un ciclo no necesariamente implica que el deadlock haya ocurrido. En este caso, un ciclo en el grafo es una condición necesaria pero no suficiente.

Ejercicio parcial

P1	P2	P3
...	...	...
Wait(x)	Wait(y)	Wait(z)
...	... ←	...
Wait(z) ←	Wait(y)	Wait(x)
...	...	...
Signal(x)		Signal(z)
...		...
Signal(z)		Signal(x)

Las flechas en cada línea indican que instrucción se está ejecutando en cada proceso. Todos los semáforos se inicializan en 1

-Dibujar grafo de alocacion. Cada semáforo es un recurso y las operaciones wait() y signal() significan *requerir* y *liberar* el recurso



-El grafo representa un estado de deadlock? Justificar

Si. Existen ciclos en el sistema:

P1 -> x -> P1 -> z -> P3 -> x -> P1 y P3 -> z -> P3 -> x -> P1 -> z -> P3

El proceso P1 esta esperando por el recurso Z, el cual lo tiene P3. El proceso P3 esta esperando por el recurso X, el cual tiene P1. Por lo tanto P1 y P3 están en deadlock.